

Dynamic Programming

Sean

August 20, 2026



Outline

- Why dynamic programming?
- The DP design process
- Your first one-dimensional DP
- Counting and optimization
- 0/1 knapsack
- Grid DP and reconstruction
- Mistakes, summary, and practice



Why dynamic programming?

- Many search trees are large because they solve the same smaller problem again and again
- Dynamic programming stores those smaller answers and reuses them
- The hard part is not writing an array: it is choosing a state that captures the right information

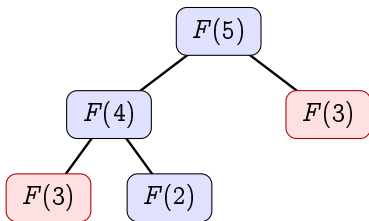


- 1 Why dynamic programming?
- 2 The DP design process
- 3 Your first one-dimensional DP
- 4 Counting and optimization
- 5 0/1 knapsack
- 6 Grid DP and reconstruction
- 7 Mistakes, summary, and practice



Why dynamic programming?

Brute force repeats work



$$F(n) = F(n - 1) + F(n - 2)$$

- The two red nodes are the same subproblem
- Their answers depend only on n , not on the path used to reach them
- Cache $F(3)$ once instead of recomputing it

Two signals that DP may fit

Overlapping subproblems

The recursive search reaches the same state from different choices.

Example: both branches above ask for $F(3)$.

Optimal substructure

An optimal answer can be assembled from optimal answers to smaller states.

Example: the best path to a cell uses a best path to its predecessor.

DP = a small state space + correct transitions between states



When is DP a good fit?

- 1 There is a natural notion of a smaller or earlier state
- 2 Each state has a manageable number of possible previous states
- 3 The answer to a state does not need the entire history, only a compact summary
- 4 The number of states times the transition work fits the limits

If the state is too big, DP is still exponential. If it is too small, the transition is wrong.

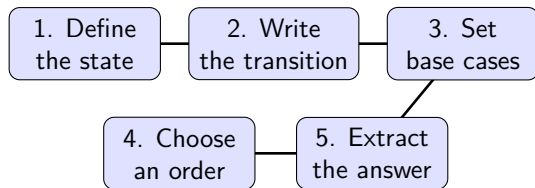


- 1 Why dynamic programming?
- 2 The DP design process
- 3 Your first one-dimensional DP
- 4 Counting and optimization
- 5 0/1 knapsack
- 6 Grid DP and reconstruction
- 7 Mistakes, summary, and practice



The DP design process

Five steps for designing a DP



The implementation comes after the recurrence is clear.

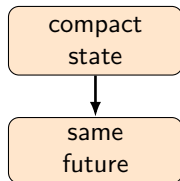
1. Define the state

Ask one precise question

What does $dp[state]$ mean?

A good state contains exactly the information the future needs.

- Stairs: $dp[i] =$ ways to reach step i
- Coins: $dp[x] =$ fewest coins to make sum x
- Grid: $dp[r][c] =$ paths to cell (r, c)

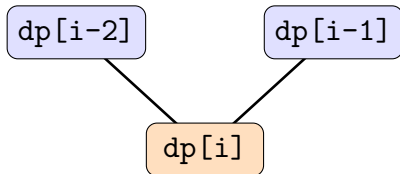


different histories
can share one answer



2. Write the transition from the last move

To reach step i , the last move was either 1 step or 2 steps.



$$dp[i] = dp[i-1] + dp[i-2]$$

3–5. Base cases, order, answer

Base cases	Start from the smallest states: $dp[0] = 1$, $dp[1] = 1$.
Evaluation order	Fill $i = 0, 1, 2, \dots, n$ so every predecessor is ready.
Extract	If the state asks for ways to reach n , the answer is $dp[n]$.
Complexity	$n + 1$ states, constant work per state: $\mathcal{O}(n)$ time and space.

A transition is only useful when its base cases and evaluation order make it executable.



- 1 Why dynamic programming?
- 2 The DP design process
- 3 Your first one-dimensional DP**
- 4 Counting and optimization
- 5 0/1 knapsack
- 6 Grid DP and reconstruction
- 7 Mistakes, summary, and practice



Your first one-dimensional DP

A complete toy problem: climbing stairs

Problem

You may climb either 1 or 2 steps at a time. How many different ways reach step n ?

- State: $dp[i] =$ ways to reach step i
- Transition: last move was from $i - 1$ or $i - 2$
- Base: $dp[0] = 1, dp[1] = 1$
- Answer: $dp[n]$



Trace: climbing stairs

1	1	2	3	5	8	13
0	1	2	3	4	5	6

1, 1, 2, 3, 5, 8, 13

- Start with $dp[0]$ and $dp[1]$
- Read two cells to write the next cell
- $dp[6] = 13$



Memoization: top-down DP

```
long long solve(int i,
                vector<long long>& memo) {
    if (i <= 1) return 1;
    if (memo[i] != -1) return memo[i];
    return memo[i] = solve(i - 1, memo) +
                    solve(i - 2, memo);
}

vector<long long> memo(n + 1, -1);
long long answer = solve(n, memo);
```

- Keep the recursive idea
- Cache an answer the first time it is computed
- Each state is solved once
- Time: $\mathcal{O}(n)$



Tabulation: bottom-up DP

```
vector<long long> dp(n + 1, 0);  
dp[0] = 1;  
if (n >= 1) dp[1] = 1;  
  
for (int i = 2; i <= n; ++i) {  
    dp[i] = dp[i - 1] + dp[i - 2];  
}  
  
long long answer = dp[n];
```

- Compute states in dependency order
- No recursion depth
- Same recurrence and same $\mathcal{O}(n)$ time
- Often easier to compress memory



Memoization versus tabulation

	Memoization	Tabulation
Control flow	Recursive, top-down	Iterative, bottom-up
States visited	Only states reached	Usually every state in the order
Main risk	Stack depth, forgotten cache	Wrong evaluation order
Good first choice	Natural recurrence or sparse states	Simple order or tight limits

Both are DP. Choose the form that makes dependencies easiest to see.



- 1 Why dynamic programming?
- 2 The DP design process
- 3 Your first one-dimensional DP
- 4 Counting and optimization**
- 5 0/1 knapsack
- 6 Grid DP and reconstruction
- 7 Mistakes, summary, and practice



Counting and optimization

Counting DP: add ways into a state

CSES: Dice Combinations

Count the ordered sequences of dice rolls whose sum is n . Each roll is 1 through 6.

- State: $dp[s]$ = number of ways to make sum s
- Last roll f gives transition $dp[s] += dp[s - f]$ for $1 \leq f \leq 6$
- Base: $dp[0] = 1$; an empty sequence makes sum 0
- Evaluate sums from small to large



Dice Combinations: a small trace

1	1	2	4	8	16	32	63
0	1	2	3	4	5	6	7

$dp[7] =$

$$32 + 16 + 8 + 4 + 2 + 1 = 63$$

- The final roll determines which earlier sum we came from
- Different last rolls describe disjoint sets of sequences
- Addition counts all valid choices exactly once



Counting DP implementation

```
1  const int MOD = 1'000'000'007;
2  vector<int> dp(n + 1, 0);
3  dp[0] = 1;
4
5  for (int sum = 1; sum <= n; ++sum) {
6      for (int face = 1; face <= 6; ++face) {
7          if (face <= sum) {
8              dp[sum] = (dp[sum] + dp[sum - face]) % MOD;
9          }
10     }
11 }
12 cout << dp[n] << '\n';
```

$\mathcal{O}(6n)$ time and $\mathcal{O}(n)$ space.



Optimization DP: choose the best predecessor

Counting

Combine all valid predecessors:

$$\text{dp}[s] = \sum \text{dp}[\text{previous}]$$

Optimization

Keep the best valid predecessor:

$$\text{dp}[s] = \text{best}(\text{candidate})$$

The state and order may stay the same; the combine operation changes.



Minimum DP: fewest coins

CSES: Minimizing Coins

Make sum x with the fewest coins. A coin may be used repeatedly.

- State: $dp[s]$ = minimum coins needed for sum s
- Try the last coin c : $dp[s] = \min(dp[s], dp[s-c] + 1)$
- Base: $dp[0] = 0$; initialize other states to INF
- If $dp[x]$ stays INF , the sum is impossible



Minimum coins: trace the choices

0	1	2	1	1	2	2
0	1	2	3	4	5	6

coins {1, 3, 4}, target 6

- $dp[3]$ becomes 1 using coin 3
- $dp[4]$ becomes 1 using coin 4
- $dp[6]$ chooses $dp[3] + 1$
- Answer: 3 + 3, two coins



Minimum-coins implementation

```
1  const int INF = 1'000'000'000;
2  vector<int> dp(x + 1, INF);
3  dp[0] = 0;
4
5  for (int sum = 1; sum <= x; ++sum) {
6      for (int coin : coins) {
7          if (coin <= sum) {
8              dp[sum] = min(dp[sum], dp[sum - coin] + 1);
9          }
10     }
11 }
12
13 cout << (dp[x] == INF ? -1 : dp[x]) << '\n';
```

$\mathcal{O}(nx)$ time and $\mathcal{O}(x)$ space.



- 1 Why dynamic programming?
- 2 The DP design process
- 3 Your first one-dimensional DP
- 4 Counting and optimization
- 5 0/1 knapsack**
- 6 Grid DP and reconstruction
- 7 Mistakes, summary, and practice



0/1 knapsack

0/1 knapsack: each item is once

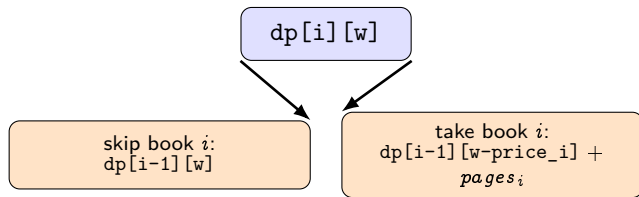
CSES: Book Shop

Pick books with total price at most x and maximize pages. Each book can be bought at most once.

- State: $dp[i][w]$ = best pages using the first i books with budget w
- For book i : skip it, or take it if its price fits
- The “at most” budget means $dp[i][w]$ can also leave money unused

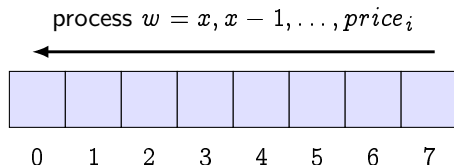


The two choices in the transition



$$dp[i][w] = \max(\text{skip}, \text{take})$$

Why the one-dimensional loop goes downward



- Descending order keeps $dp[w - price]$ from the previous item
- Ascending order can reuse the same item many times
- This is the difference between 0/1 knapsack and unbounded knapsack

0/1 knapsack: memory-compressed code

```
1  vector<int> dp(capacity + 1, 0);
2
3  for (int i = 0; i < n; ++i) {
4      for (int w = capacity; w >= price[i]; --w) {
5          dp[w] = max(dp[w],
6                      dp[w - price[i]] + pages[i]);
7      }
8  }
9
10 cout << dp[capacity] << '\n';
```

$\mathcal{O}(nx)$ time and $\mathcal{O}(x)$ space.



Book Shop: a worked example

book i	1	2	3	4
price	4	8	5	3
pages	5	12	8	1
capacity:	10		best:	13
1	2	3	4	

- Take books 1 and 3
- Price
 $4 + 5 = 9 \leq 10$
- Pages $5 + 8 = 13$
- The DP checks every legal skip/take sequence

- 1 Why dynamic programming?
- 2 The DP design process
- 3 Your first one-dimensional DP
- 4 Counting and optimization
- 5 0/1 knapsack
- 6 Grid DP and reconstruction**
- 7 Mistakes, summary, and practice



Grid DP and reconstruction

Grid DP: the state is a cell

CSES: Grid Paths I

Count paths from the upper-left to the lower-right of a grid. Move only right or down; traps are blocked.

- State: $dp[r][c] =$ paths to (r, c)
- A path enters from above or from the left
- A trap has value 0

start

1	1	1	1
1	*	1	2
1	1	2	*
*	1	3	3

answer = 3



Grid transitions are local

For an open cell (r, c) :

$$dp[r][c] = dp[r-1][c] + dp[r][c-1]$$

- Handle the first row and first column through bounds checks
- Set a trap to 0 before applying the transition
- Fill row by row: top and left neighbors are already ready
- Modulo after additions when the number of paths can be huge



Grid DP implementation

```
1  const int MOD = 1'000'000'007;
2  vector<vector<int>> dp(n, vector<int>(n));
3
4  for (int r = 0; r < n; ++r) {
5      for (int c = 0; c < n; ++c) {
6          if (grid[r][c] == '*') continue;
7          if (r == 0 && c == 0) dp[r][c] = 1;
8          else {
9              if (r > 0) dp[r][c] = dp[r - 1][c];
10             if (c > 0) dp[r][c] = (dp[r][c] +
11                                     dp[r][c - 1]) % MOD;
12         }
13     }
14 }
15 cout << dp[n - 1][n - 1] << '\n';
```



$\mathcal{O}(n^2)$ time and $\mathcal{O}(n^2)$ space.

DP can remember choices, not just values

Value DP

Stores the best/count value for every state.

Example: maximum pages for each budget.

Decision DP

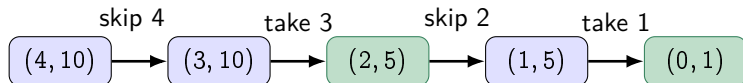
Also store whether the winning transition took an item or moved from above/left.

Then walk backward from the final state.

Reconstruction is a second pass over already-computed states.



Backtracking the Book Shop choice



- At (i, w) , compare $dp[i][w]$ with $dp[i-1][w]$
- Equal means “skip i ”; otherwise the optimal transition took i
- The chosen books are 1 and 3, in reverse discovery order



Reconstruction code pattern

```
1  vector<int> chosen;
2  int w = capacity;
3
4  for (int i = n; i >= 1; --i) {
5      if (take[i][w]) {
6          chosen.push_back(i);
7          w -= price[i];
8      }
9  }
10 reverse(chosen.begin(), chosen.end());
```

Store a take decision while filling a 2D table, or recompute the comparison during backtracking.



- 1 Why dynamic programming?
- 2 The DP design process
- 3 Your first one-dimensional DP
- 4 Counting and optimization
- 5 0/1 knapsack
- 6 Grid DP and reconstruction
- 7 Mistakes, summary, and practice



Mistakes, summary, and practice

Common DP mistakes

- State is missing information: two histories with different futures are merged
- State stores too much information: the number of states is still exponential
- Base cases are inconsistent with the transition
- Evaluation order reads an answer before it has been computed



More mistakes to catch early

- Reusing a 0/1 item by looping capacity upward
- Treating an unreachable state as a real value instead of INF or $-INF$
- Forgetting modulo arithmetic or using an integer type that overflows
- Reconstructing from a compressed 1D table without retaining enough decisions
- Assuming a greedy choice works just because the DP transition looks local

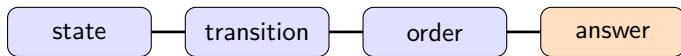


A small-test debugging checklist

- 1 Draw the state graph for the smallest non-trivial input
- 2 Write the first few DP values by hand
- 3 Check every transition against the words in your state definition
- 4 Test an impossible case, an empty/base case, and a boundary case
- 5 Only then optimize memory or code length



The DP recipe in one picture



- Define what one entry means
- Derive it from smaller states
- Make prerequisites ready before each update
- Add, minimize, or maximize exactly what the problem asks for



Practice roadmap

Start here

- [Dice Combinations](#) – counting
- [Minimizing Coins](#) – minimum
- [Removing Digits](#) – one-dimensional choices
- [Book Shop](#) – 0/1 knapsack

Then stretch

- [Grid Paths I](#) – grid DP
- [Array Description](#) – state with a neighbor value

Future topics

- digit DP, bitmask DP, tree DP, DAG DP



Thank you

Thank you.

© 2026 Sean. Please do not repost or redistribute without permission.

